Manual Técnico

Market Express



Versión 1.0 – octubre 2026

Arquitectura: Cliente-Servidor de 3 Capas (MVC simplificado)

Stack de Desarrollo: LAMP / XAMPP (Apache 2.4, MariaDB 10.4+, PHP 8.1+)

Lenguajes Lado Servidor: PHP 8.x (Programación Orientada a Objetos / PDO)

Lenguajes Lado Cliente: HTML5, CSS3 (CSS Variables, Flexbox/Grid), JavaScript (ES6+ Native)

Librerías de Terceros: Html5-Qrcode API (v2.3.8) para escaneo por cámara

Motor de BD y Cotejamiento: InnoDB (utf8mb4_unicode_ci)

Índice de contenido

Contenidos

Requerimientos Técnicos Del Entorno 1

Estructura Y Árbol De Directorios Del Proyecto 2

Ddl Completo Y Esquema Relacional (Mysql / Innodb) 3

Explicación Detallada de las Tablas del Sistema 9

Lógica De Detección Dinámica De Ip En Php..... 14

Arquitectura De Seguridad Y Protección De Dato 16

Especificación De Endpoints De La Api Rest..... 18

Configuración De Impresión Térmica 20

Plan De Mantenimiento Y Respaldos (Backups)..... 21

Requerimientos Técnicos Del Entorno

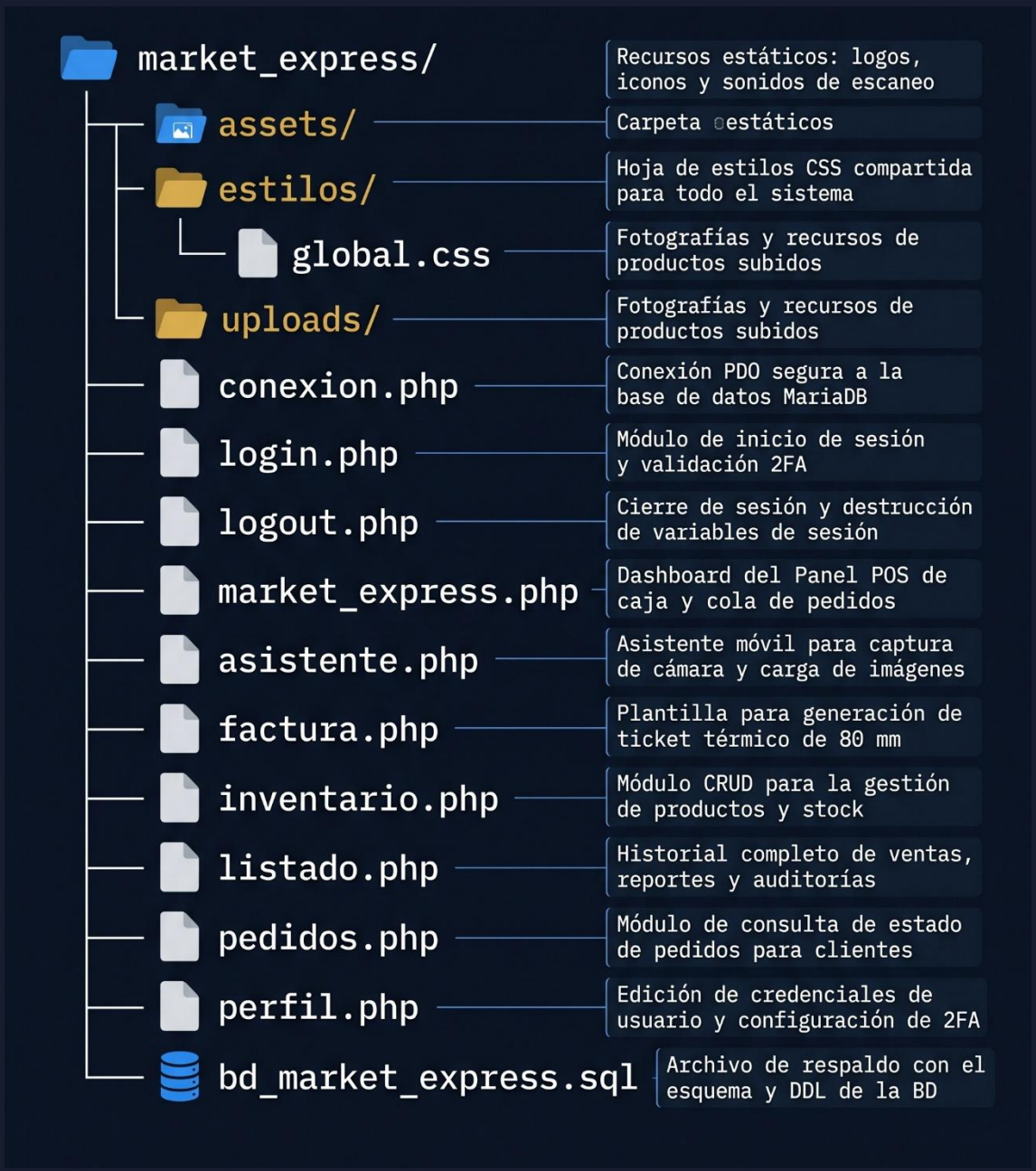
Requerimientos Mínimos del Servidor Web (Host / PC Caja)

- **Procesador:** x86/x64 dual-core a 2.0 GHz o superior.
- **Memoria RAM:** 4 GB mínimo (8 GB recomendado).
- **Almacenamiento:** 500 MB libres en disco SSD/HDD.
- **Sistema Operativo:** Windows 10/11 (64-bit) o GNU/Linux (Ubuntu Server/Desktop 20.04 LTS o superior).
- **Entorno Ejecutable:** XAMPP v8.1+ con extensiones PHP activas: pdo_mysql, gd, session, json, mbstring.

Requerimientos de Cliente y Dispositivo Móvil

- **Navegador Web:** Google Chrome 100+, Mozilla Firefox 98+ o Safari 15+ con soporte para MediaDevices API (Cámara HD).
- **Cámara Trasera:** Resolución mínima de 2 MP con enfoque automático para la lectura fluida de códigos de barras/QR en app_ia.php.
- **Red:** Interfaz Wi-Fi IEEE 802.11 b/g/n en la misma subred IPv4 que el servidor de caja.

Estructura Y Árbol De Directorios Del Proyecto



Ddl Completo Y Esquema Relacional (Mysql / Innodb)

```
CREATE DATABASE IF NOT EXISTS `market_express`
DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE `market_express`;

-- 1. TABLA DE CATEGORÍAS
CREATE TABLE IF NOT EXISTS `categorias` (
  `id` INT(11) NOT NULL AUTO_INCREMENT,
  `nombre` VARCHAR(50) NOT NULL,
  `icono` VARCHAR(50) DEFAULT 'box',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
```

```
-- 2. TABLA DE PRODUCTOS
CREATE TABLE IF NOT EXISTS `productos` (
  `id` INT(11) NOT NULL AUTO_INCREMENT,
  `categoria_id` INT(11) NOT NULL,
  `codigo` VARCHAR(50) NOT NULL,
  `nombre` VARCHAR(100) NOT NULL,
  `precio` DECIMAL(10,2) NOT NULL,
  `descuento` DECIMAL(5,2) DEFAULT 0.00,
  `stock` INT(11) NOT NULL DEFAULT 0,
  `imagen` VARCHAR(255) DEFAULT 'default.png',
  `descripcion` TEXT DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_codigo` (`codigo`),
  KEY `fk_productos_categorias` (`categoria_id`),
  CONSTRAINT `fk_productos_categorias`
    FOREIGN KEY (`categoria_id`) REFERENCES `categorias`
(`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
```

```
-- 3. TABLA DE USUARIOS (CLIENTES)
CREATE TABLE IF NOT EXISTS `usuarios` (
  `id` INT(11) NOT NULL AUTO_INCREMENT,
  `nombre_completo` VARCHAR(120) NOT NULL,
  `correo` VARCHAR(100) NOT NULL,
  `password` VARCHAR(255) NOT NULL,
  `fecha_registro` TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  `foto` VARCHAR(255) DEFAULT 'default_user.png',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_correo_usuario` (`correo`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;

-- 4. TABLA DE PEDIDOS Y VENTAS
CREATE TABLE IF NOT EXISTS `pedidos` (
  `id` INT(11) NOT NULL AUTO_INCREMENT,
  `usuario_id` INT(11) DEFAULT NULL,
  `subtotal` DECIMAL(10,2) NOT NULL,
  `descuento` DECIMAL(10,2) DEFAULT 0.00,
  `impuesto` DECIMAL(10,2) NOT NULL,
  `total` DECIMAL(10,2) NOT NULL,
```

```
  `efectivo_recibido` DECIMAL(10,2) NOT NULL,
  `cambio` DECIMAL(10,2) NOT NULL,
  `estado` ENUM('pendiente','completado','cancelado') NOT
NULL DEFAULT 'pendiente',
  `fecha` TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `fk_pedidos_usuarios` (`usuario_id`),
  CONSTRAINT `fk_pedidos_usuarios`
    FOREIGN KEY (`usuario_id`) REFERENCES `usuarios` (`id`)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;

-- 5. TABLA INTERMEDIA DETALLE DE PEDIDOS
CREATE TABLE IF NOT EXISTS `detalle_pedidos` (
  `id` INT(11) NOT NULL AUTO_INCREMENT,
  `pedido_id` INT(11) NOT NULL,
  `producto_id` INT(11) NOT NULL,
  `cantidad` INT(11) NOT NULL,
  `precio_unitario` DECIMAL(10,2) NOT NULL,
  `subtotal` DECIMAL(10,2) NOT NULL,
```

```
  PRIMARY KEY (`id`),
  KEY `fk_detalle_pedidos` (`pedido_id`),
  KEY `fk_detalle_productos` (`producto_id`),
  CONSTRAINT `fk_detalle_pedidos`
    FOREIGN KEY (`pedido_id`) REFERENCES `pedidos` (`id`)
    ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT `fk_detalle_productos`
    FOREIGN KEY (`producto_id`) REFERENCES `productos`
(`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;

-- 6. TABLA DE ADMINISTRADORES
CREATE TABLE IF NOT EXISTS `administradores` (
  `id` INT(11) NOT NULL AUTO_INCREMENT,
  `nombre_completo` VARCHAR(120) NOT NULL,
  `correo` VARCHAR(100) NOT NULL,
  `password` VARCHAR(255) NOT NULL,
  `cargo` VARCHAR(50) DEFAULT 'Cajero',
  `fecha_registro` TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
```

```
  `foto` VARCHAR(255) DEFAULT 'default_admin.png',
  `pin_2fa` VARCHAR(255) DEFAULT NULL,
  `estado_2fa` TINYINT(1) DEFAULT 0,
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_correo_admin` (`correo`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;

-- 7. TABLA DE HISTORIAL Y AUDITORÍA
CREATE TABLE IF NOT EXISTS `historial_admin` (
  `id` INT(11) NOT NULL AUTO_INCREMENT,
  `admin_id` INT(11) NOT NULL,
  `accion` VARCHAR(255) NOT NULL,
  `fecha` TIMESTAMP NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  KEY `fk_historial_admin` (`admin_id`),
  CONSTRAINT `fk_historial_admin`
    FOREIGN KEY (`admin_id`) REFERENCES `administradores`
(`id`)
    ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4
COLLATE=utf8mb4_unicode_ci;
```

Explicación Detallada de las Tablas del Sistema

1. categorias

- **Propósito:** Clasifica y agrupa los productos del comercio en familias o secciones (ejemplo: *Lácteos, Granos Básicos, Bebidas, Enlatados*).
- **Funcionamiento:** Sirve como tabla maestra para estructurar los filtros de búsqueda tanto en el catálogo de la tienda web del cliente como en la cuadrícula de selección rápida del panel de caja (POS).

2. productos

- **Propósito:** Almacena el catálogo completo de inventario con sus precios, códigos e imágenes.
- **Funcionamiento:**
    - Se conecta con categorias mediante la llave foránea categoria_id.
    - Guarda el codigo único de barras o QR utilizado por el asistente móvil (asistente.php) para el escaneo en caja.

- Gestiona el stock disponible en tiempo real; cada vez que se finaliza una venta, la base de datos descuenta las unidades correspondientes para prevenir sobreventas.
- Almacena el porcentaje de descuento aplicable individualmente a cada artículo.

3. usuarios

- **Propósito:** Gestiona las cuentas de los clientes finales que acceden a la plataforma desde la tienda web.
- **Funcionamiento:** Guarda los datos personales (nombre, correo) y las credenciales de acceso. La contraseña (password) se almacena cifrada mediante el algoritmo **BCRYPT** para garantizar la privacidad y la protección de datos.

4. pedidos

- **Propósito:** Funciona como la cabecera principal de cada transacción comercial, ya sea una orden solicitada desde la web por un cliente o una venta presencial directa en caja.
- **Funcionamiento:**

- Se vincula opcionalmente con usuarios mediante usuario_id (si el usuario está registrado, o NULL si es un cliente general de mostrador).
- Consolida los valores globales de la venta: subtotal, descuento, impuesto (IVA 13%), total, efectivo_recibido y cambio entregado.
- Controla el ciclo de vida de la transacción a través de la columna estado (*pendiente, completado, cancelado*).

5. detalle_pedidos

- **Propósito:** Resuelve la relación de muchos a muchos ($N:M$) entre las ventas (pedidos) y los artículos del inventario (productos).
- **Funcionamiento:**
  - Un único pedido puede contener múltiples productos, y un producto puede figurar en muchos pedidos históricos.
  - Registra el desglose ítem por ítem de la venta: la cantidad comprada, el precio unitario cobrado al momento de la transacción y el subtotal por renglón.
  - Gracias a ON DELETE CASCADE, si un pedido se cancela o elimina, todos sus renglones de detalle se limpian automáticamente para mantener la base de datos optimizada.

6. administradores

- **Propósito:** Gestiona el control de acceso para cajeros, supervisores y administradores del punto de venta.
- **Funcionamiento:**
    - Almacena las credenciales administrativas con contraseñas cifradas.
    - Contiene las columnas pin_2fa y estado_2fa para habilitar la **Autenticación en Dos Pasos (2FA)**, exigiendo un PIN de seguridad secundario antes de permitir el acceso al panel de caja o al inventario.
    - Define el rol o cargo del usuario para la asignación de permisos dentro del sistema.

7. historial_admin

- **Propósito:** Módulo de auditoría e inspección interna.
- **Funcionamiento:**
    - Se vincula con administradores mediante admin_id.
    - Registra de manera cronológica cada acción crítica realizada dentro del sistema (ejemplo: *"Creación de nuevo producto ID #12"*, *"Venta en caja por $25.00"*, *"Modificación de precio"*).
    - Incluye la marca de tiempo exacta (fecha) para garantizar la transparencia operativa y la rendición de cuentas del personal.

🔄 Flujo de Datos en una Venta Real

1. **Lectura/Escaneo:** El asistente móvil o el cajero escanea el código en asistente.php, lo que realiza una consulta SELECT a la tabla productos.
2. **Procesamiento de Venta:** Al presionar "Finalizar Venta", el backend inicia una transacción PDO en MySQL:
    - Inserta el registro de cabecera en pedidos con los totales y montos de efectivo.
    - Obtiene el ID del pedido recién generado e inserta cada ítem vendido dentro de detalle_pedidos.
    - Ejecuta una consulta UPDATE sobre la tabla productos reduciendo el stock actual según la cantidad vendida.
    - Registra automáticamente la operación en la tabla historial_admin con el ID del cajero activo.

Lógica De Detección Dinámica De Ip En Php

Para evitar que el modal de vinculación genera un código QR apontando a adaptadores virtuales (como VirtualBox, WSL o interfaces *loopback*), la función filtra dinámicamente el listado de interfaces devuelto por el sistema operativo:

```
function obtenerIpRedLocal() {
    $ipValida = '127.0.0.1';

    // Ejecución de comandos del sistema según SO
    if (strtoupper(substr(PHP_OS, 0, 3)) === 'WIN') {
        $output = shell_exec('ipconfig');
        preg_match_all('/IPv4[^\:]*:\s*([0-9\.]+)/', $output, $matches);
        $ips = $matches[1] ?? [];
    } else {
        $output = shell_exec("hostname -I 2>/dev/null || ip addr");
        preg_match_all('/(192\.168\.[0-9]+\.[0-9]+|10\.[0-9]+\.[0-9]+\.[0-9]+)/', $output, $matches);
        $ips = $matches[1] ?? [];
```

```
    }

    foreach ($ips as $ip) {
        $ip = trim($ip);

        // Exclusión estricta de rangos virtuales y loopback
        if ($ip === '127.0.0.1') continue;
        if (strpos($ip, '192.168.56.') === 0) continue; // VirtualBox Host-Only
        if (strpos($ip, '172.16.') === 0) continue;     // Adaptadores Docker/VM/WSL
        if (strpos($ip, '169.254.') === 0) continue;    // Direcciones APIPA

        // Priorizar el rango LAN estándar (192.168.x.x o 10.x.x.x)
        if (strpos($ip, '192.168.') === 0 || strpos($ip, '10.') === 0) {
            return $ip;
        }
    }

    return $ipValida;
}
```

Arquitectura De Seguridad Y Protección De Dato

```
// Ejemplo de consulta PDO en conexion.php
class Conexion {
   private static $instancia = null;

   public static function getConexion() {
      if (self::$instancia === null) {
         try {
            self::$instancia = new PDO(

"mysql:host=localhost;dbname=market_express;charset=utf8mb4",
               "root",
               "",
               [
                  PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                  PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
```

```
                PDO::ATTR_EMULATE_PREPARES => false
            ]
        );
    } catch (PDOException $e) {
        die(json_encode(['status' => 'error', 'msg' => 'Fallo de conexión a la BD']));
    }
  }
  return self::$instancia;
 }
}
```

Hashing de Contraseñas y Verificación 2FA

- **Almacenamiento:** Uso obligatorio del algoritmo BCRYPT mediante la función password_hash($pass, PASSWORD_BCRYPT).
- **Verificación:** Comparación segura de Hashes mediante password_verify($pass_ingresada, $hash_bd).
- **2FA (PIN):** Cifrado hash del PIN de 6 dígitos en la columna pin_2fa. Si estado_2fa = 1, la sesión requiere paso de validación previo a la emisión del token de acceso $_SESSION['admin_autenticado'].

Especificación De Endpoints De La Api Rest

El asistente móvil interactúa de forma asíncrona mediante peticiones HTTP POST formateadas en JSON:

Endpoint: POST /market_express/api_ia.php

Solicitud 1: Procesar código escaneado por cámara


```
{
  "accion": "agregar_codigo",
  "codigo": "7411000852014"
}
```


Respuesta de Éxito (200 OK):


```
{
  "status": "success",
  "mensaje": "Producto agregado a la caja",
  "producto": {
    "id": 4,
    "nombre": "Leche Entera 1L",
```

```
  "precio": 1.75,
  "stock": 48
 }
}
```


Respuesta de Error (400 / 404):


```
{
 "status": "error",
 "mensaje": "El código no corresponde a ningún producto registrado"
}
```

Configuración De Impresión Térmica

Para asegurar que los comprobantes se adapten a impresoras térmicas de 80 mm sin desbordamiento de página, el módulo utiliza CSS @media print:

```
@media print {
  @page {
    margin: 0;
    size: 80mm auto;
  }
  body {
    width: 78mm;
    margin: 0 auto;
    font-family: 'Courier New', Courier, monospace;
    font-size: 11px;
    color: #000;
    background: #fff;
  }
```

```
    .no-printable {
        display: none !important;
    }
}
```

Plan De Mantenimiento Y Respaldos (Backups)

Respaldo Automatizado de Base de Datos (Script CLI): Para programar respaldos periódicos en Windows (Programador de Tareas) o Linux (Cron Job):

```
mysqldump -u root -p market_express > C:\backups\market_express_%date:~-4%-%date:~3,2%-%date:~0,2%.sql
```

Mantenimiento de Tablas InnoDB: Ejecutar trimestralmente desde phpMyAdmin o consola para optimizar índices y desfragmentar espacio:

```
OPTIMIZE TABLE productos, pedidos, detalle_pedidos, historial_admin;
```